

SFA2 — État de la détection pixel & isolation

KOF98

Date : 2026-07-14

Branche : `main` (27 fichiers modifiés non commités)

Contexte : Détection pixel pour SFA2 (SNES/snes9x — RAM inaccessible), sans casser KOF98 (RAM Neo Geo/FBNeo)

1. Architecture des deux chemins de détection

```
GameRunner.start()
├─ healthMemMap? (RAM config présente ?)
│   └─ OUI → startMemoryHealthReader()
│       └─ UDP_READ_CORE_RAM → processHealthFrame()
│           └─ p1Lost != null? → processLossCounters() → RETURN ← KOF98
│               └─ sinon → fallback santé heuristique (legacy)
└─ NON → startHealthBarCapture()
    └─ ffmpeg x11grab → analyzeHealthFrame() ← SFA2, SF2, ...
        state machine: WARMUP→PLAYING→KO_PENDING→KO_CONFIRMED→MATCH_END
```

Les deux chemins sont complètement séparés au niveau de la détection.

Ils partagent uniquement des variables de sortie : `p1Losses` , `p2Losses` , `matchEnded` , `roundNumber` , `matchPerfectKos` .

2. KOF98 — Chemin RAM (FBNeo) : PROTÉGÉ

Ce qui est en place

Détection	Mécanisme	Adresse(s) RAM	Statut
Santé P1/P2	UDP <code>READ_CORE_RAM</code>	<code>0x8238</code> / <code>0x8438</code>	 Stable
Timer 16-bit	UDP (timer-only fast poll)	<code>0xA83A-A83B</code>	
Perso actif P1/P2	Lecture RAM	<code>0x8256</code> / <code>0x8456</code>	
Mode gauge	Lecture RAM	<code>0x821E</code> / <code>0x841E</code>	 Adresses pas encore validées
Équipes (3 persos)	Lecture RAM + freeze	<code>0xA84E</code> / <code>0xA84F</code> / <code>0xA851</code> (P1)	 Figées au combat
Ordre de sélection	Lecture RAM one-shot	<code>0x15CB</code> / <code>0x15CA</code> / <code>0x15CD</code> (P1)	 Capturé, pas encore câblé dans overlay
Compteur de pertes	Autoritaire	<code>0xA859</code> (P1) / <code>0xA868</code> (P2)	 C'est ça qui drive tout
Round gagné/perdu	<code>processLossCounters()</code>	Diff <code>prevLost</code> → <code>currLost</code>	
Time-over	<code>memTimer16 > 0 → 0</code> transition	<code>0xA83A</code>	

Détection	Mécanisme	Adresse(s) RAM	Statut		
KO	Perte incrémentée sans time-over	Compteurs	✓		
Perfect KO	<code>roundMinHealth ≥ 95%</code> (combat stable)	Heuristique (pas de compteur RAM fiable)	✓		
Draw	Les deux compteurs incrémentés au même poll	<code>0xA859 + 0xA868</code>	✓		
Fin de match	<code>p1Lost ≥ 3</code>		<code>p2Lost ≥ 3`</code>	Hardcodé à 3	✓
Continue/Revanche	Pièce via <code>0xF2C0</code> , START loop	UDP <code>btn</code> commands	✓		

Pourquoi KOF98 n'est pas affecté par les changements SFA2

- `processHealthFrame()` (ligne 1422) : Si `healthMemMap?.p1Lost != null` , appelle `processLossCounters()` et **return immédiatement** — le code pixel n'est jamais atteint
- `analyzeHealthFrame()` est appelé uniquement depuis le handler `ffmpeg.stdout.on("data")` — qui n'est jamais démarré pour KOF98 (car `startMemoryHealthReader()` est appelé à la place)
- `pixelConfig` vaut `null` pour KOF98 (`getPixelConfig("kof98.zip")`) → pas d'entrée dans `PIXEL_GAME_CONFIGS`)
- **Match end KOF98** : `processLossCounters()` ligne 1397 → `p1Lost >= 3 || p2Lost >= 3` (hardcodé, unchanged)

⚠ Points d'attention KOF98

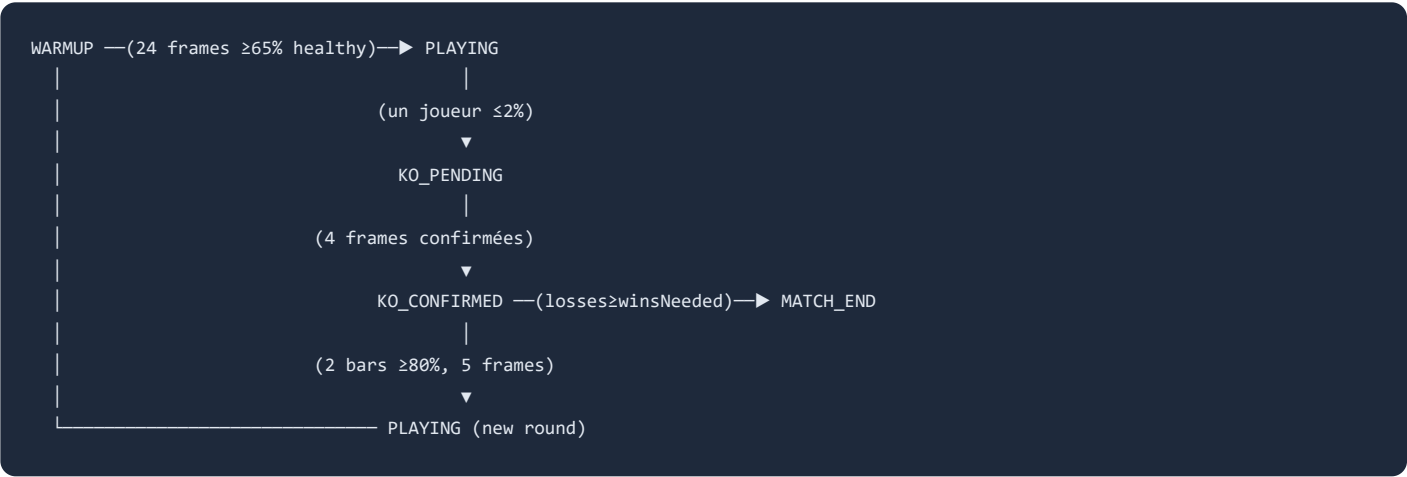
- **Mode gauge ADVANCED/EXTRA** : les adresses `0x81F0 / 0x83F0` étaient fausses, remplacées par `0x821E / 0x841E` dans la config DB — **pas encore validées live**
- **Ordre de sélection** : adresses trouvées et validées (`0x15CB / 0x15CA / 0x15CD`), mais **pas encore câblé** dans `matchMeta()` pour l'overlay de fin de match
- `winsNeeded` **hardcodé à 3** dans `processLossCounters()` — si un jour on veut du KOF98 en BO1, il faudra paramétrer

3. SFA2 — Chemin pixel (snes9x) : EN COURS

Config par ROM

```
PIXEL_GAME_CONFIGS = {
  "Street Fighter Alpha 2 (Europe).sfc": {
    stripeY: 110, stripeH: 24,    // barre de vie = 24px à y=110 (après bordure noire)
    p1StartX: 70, p1EndX: 310,    // barre P1 à gauche
    p2StartX: 450, p2EndX: 768,    // barre P2 à droite
    winsNeeded: 2,                // best-of-3 (SF2 rules)
    timer: {
      digits: [bitmaps 0-9],      // templates 8x12 bits
      leftDigitX: 338, rightDigitX: 362,
      digitW: 22, digitH: 24,
      minBrightRatio: 0.15,
    },
  },
};
```

State machine (GamePhase)



Ce qui est fait (dans le code non commité)

Détection	Mécanisme	Fichier	Statut
Santé P1/P2	Column scan + saturation couleur	game-runner.ts:analyzeHealthFrame()	 Implémenté
Barre calibrée	p1FullBarWidth / p2FullBarWidth max pendant WARMUP	game-runner.ts	
Lissage	Médiane glissante sur 5 frames	game-runner.ts:getSmoothedHealth()	

Détection	Mécanisme	Fichier	Statut
Seuil KO	<code>KO_THRESHOLD = 2%</code>	<code>game-runner.ts</code>	✓
Confirmation KO	4 frames consécutives (<code>KO_CONFIRM_REQUIRED</code>)	<code>game-runner.ts</code>	✓
Grâce début round	16 frames après entrée PLAYING (<code>PLAYING_GRACE_FRAMES</code>)	<code>game-runner.ts</code>	✓
KO	<code>KO_PENDING</code> → <code>KO_CONFIRMED</code> , détermine vainqueur	<code>game-runner.ts:analyzeHealthFrame()</code>	✓
Perfect KO	<code>roundP1MinHealth ≥ 95%</code> OU <code>roundP2MinHealth ≥ 95%</code>	<code>game-runner.ts</code>	✓
Draw	Les deux joueurs KO simultanément, santé égale	<code>game-runner.ts</code>	✓
Nouveau round	2 barres ≥ 80% pendant 5 frames → retour PLAYING	<code>game-runner.ts</code>	✓
Fin de match	<code>losses ≥ winsNeeded</code> (2 pour SFA2)	<code>game-runner.ts</code>	✓
Timer OCR	Template matching 8×12 → digits 0-99	<code>game-runner.ts:readTimerFromFrame()</code>	✓
Validation timer	Stabilité N frames, transitions valides	<code>game-runner.ts:processTimerValue()</code>	✓
Config par ROM	<code>PIXEL_GAME_CONFIGS</code> + <code>getPixelConfig()</code>	<code>game-runner.ts</code>	✓
Warmup rapide	<code>fastWarmup = true</code> → 8 frames au lieu de 24	<code>game-runner.ts:resetHealthWarmup()</code>	✓
Anti-faux-positif	Double-drop simultané ignoré (transition écran)	<code>game-runner.ts</code>	✓

Ce qui MANQUE pour SFA2

Détection	Problème	Priorité
Time-over	Le timer est LU (OCR → digits) mais jamais utilisé pour détecter une fin de round. <code>processTimerValue()</code> ne fait que valider et logger. Il faut : détecter <code>timer → 0</code> dans le state <code>PLAYING</code> , comparer les santés restantes, émettre <code>roundResult</code> avec <code>koType: "timeout"</code> .	 URGENT
Émission roundResult	L'event <code>roundResult</code> est émis dans <code>KO_CONFIRMED</code> — mais il faut aussi l'émettre sur time-over.	
Tests live SFA2	Tout le code pixel est théorique — nécessite des vrais combats SFA2 pour calibrer les seuils.	
Overlay SFA2	Pas de <code>charInfo()</code> pertinente pour SFA2 (pas de RAM → pas de noms de persos). L'overlay de fin de match sera minimal.	
Autres jeux SNES	La structure <code>PIXEL_GAME_CONFIGS</code> est prête à accueillir d'autres ROMs. Ajouter SF2, Killer Instinct, etc. = juste une entrée dans le map.	
Config charger depuis DB	Actuellement <code>PIXEL_GAME_CONFIGS</code> est hardcodé. Le game-server a maintenant les vars Turso — on pourrait charger la config pixel depuis <code>duel_games.ram_config</code> .	

4. Reste du chantier (autres fichiers modifiés)

DB & Config — `db.ts`

- ✓ Tables `duel_games` , `duel_game_modes` , `duel_game_controls` , `duel_game_config_versions`
- ✓ Seed 4 jeux × 3 modes (standard/XL/fighter)
- ✓ Règles multilingues FR/EN
- ✓ RAM config KOF98 + KOF2002 stockée en JSON
- ⚠ Config pixel SFA2 pas encore en DB (hardcodé dans `PIXEL_GAME_CONFIGS`)

Flow règles — `respond/route.ts` + `confirm-rules/route.ts`

- ✓ Accept → `rules_pending` (ne crée plus la session directement)
- ✓ Nouvelle route `confirm-rules` (fichier untracked)
- ✓ Notifications aux deux joueurs

- ☒ Colonnes `challenger_rules_accepted` / `target_rules_accepted`

UI Duel — `DuelLobby.tsx` , `DuelScoreHUD.tsx` , etc.

- ☒ Filtre joueurs, lien invitation, recovery après refresh
- ☒ Composants : `DuelScoreHUD` , `DuelRulesOverlay` , `DuelPauseOverlay` , `DuelGameSelector` , `DuelModeSelector` , `DuelWizardStepper`
- ☒ i18n étendu (sections rules, pause, wizard, mode)
- ☒ Frais d'entrée dynamiques par jeu/mode

game-server — `config.ts` , `docker-compose`

- ☒ Support SNES (core snes9x, boutons, résolution)
- ☒ Variables Turso dans Docker

5. Plan d'action recommandé

Étape 1 — Time-over SFA2 (le vrai trou)

```
Dans analyzeHealthFrame(), état PLAYING :
- Lire le timer via readTimerFromFrame()
- Si lastTimerValue passe de >0 à 0 :
  → comparer p1Health vs p2Health
  → émettre roundResult avec koType: "timeout"
  → incrémenter p1Losses ou p2Losses
  → passer en KO_CONFIRMED
```

Étape 2 — Valider live

- Lancer un combat SFA2, laisser le timer s'épuiser
- Vérifier que le time-over est bien détecté
- Vérifier KO, perfect KO, draw

Étape 3 — Nettoyer et committer

- Une fois la détection SFA2 complète, committer par lots logiques
- Ne pas mélanger les changements game-runner avec les changements UI

Étape 4 — KOF98 gauge mode

- Valider les adresses `0x821E` / `0x841E` par diff live (ADVANCED vs EXTRA)

6. Résumé visuel

```

GameRunner.start()

healthMemMap? (RAM config from DB or hardcoded)
├── OUI (KOF98, KOF2002)
│   ├── startMemoryHealthReader()
│   │   └── UDP READ_CORE_RAM every 250ms
│   │       ↓
│   │   processHealthFrame()
│   │       ├── p1Lost != null? → processLossCounters()
│   │       │   ├── ✓ Rounds ✓ KO ✓ Time-over
│   │       │   ├── ✓ Perfect ✓ Draw ✓ Match end
│   │       │   └── ⚠ Gauge mode (adresses à valider)
│   └── NON (SFA2, SF2, autres SNES)
│       ├── startHealthBarCapture()
│       │   └── ffmpeg x11grab 2fps → stripe santé
│       │       ↓
│       │   analyzeHealthFrame()
│       │       └── State machine:
│       │           ├── ✓ Santé (column scan + médiane)
│       │           ├── ✓ KO (4-frame confirm)
│       │           ├── ✓ Perfect KO (minHealth ≥ 95%)
│       │           ├── ✓ Draw (double KO simultané)
│       │           ├── ✓ Nouveau round (2 barres ≥ 80%)
│       │           ├── ✓ Match end (winsNeeded configurable)
│       │           ├── ✓ Timer OCR (digits lus)
│       │           └── ✗ Time-over (timer pas utilisé!)
└──
```